

Python Network Automation: Pocket Reference

Configuring Networks with Object-Oriented, Security-Scripting Style

Scope: Sections 1–6 from `nesc.txt`

Goal: Learn the *idea*, the *workflow*, and a *minimal working example* for each network automation tool.

Style: Object-oriented Python, with security-scripting flavoured examples (devices, commands, configs, backups).

1. The Big Picture

Task	What it does	Real-life security example
Network Automation	Use code to configure, monitor, and manage network devices	Push ACL changes to 50 routers, back up switch configs nightly, audit interface status.
Programmatic Access	Replace manual CLI typing with repeatable scripts	SSH to a firewall, collect <code>show version</code> , parse for vulnerable firmware.

Mental model

```
Inventory (devices) → Driver/Tool (SSH/API) → Commands/Config → Verification
routers.yml          Paramiko/Netmiko        show / configure      diff /
check
```

2. What Is Network Automation?

Network automation means using software to perform network operations without logging into each device manually.

Typical workflow

1. **Inventory** — list devices (IP, vendor, credentials).
2. **Connect** — SSH, Telnet, NETCONF, RESTCONF, SNMP, or API.
3. **Execute** — send `show` commands or configuration commands.
4. **Parse** — turn raw text into structured data.
5. **Act** — save configs, generate reports, open tickets, or rollback.

```
class NetworkTask:
    def __init__(self, device_ip, username, password):
```

```

        self.device_ip = device_ip
        self.username = username
        self.password = password

    def run(self):
        # Subclasses implement the actual connection logic
        raise NotImplementedError("Use a concrete driver like SshTask")

```

3. Types of Network Automation

Type	Description	Example
Configuration management	Push or pull device configs	Update VLANs across all access switches.
Data collection / monitoring	Gather state and statistics	Collect interface errors every 5 minutes.
Compliance / auditing	Verify devices match a golden standard	Ensure every router has <code>ntp server 10.0.0.1</code> .
Provisioning / orchestration	Build new networks from templates	Spin up a branch office with DHCP, DNS, and routing.
Event-driven / self-healing	React to telemetry in real time	Shut down a port when a loop is detected.

```

class AuditTask(NetworkTask):
    def __init__(self, device_ip, username, password, golden_rules):
        super().__init__(device_ip, username, password)
        self.golden_rules = golden_rules

    def check(self, running_config):
        findings = []
        for rule in self.golden_rules:
            if rule not in running_config:
                findings.append(f"MISSING: {rule}")
        return findings

# Example: audit SSH-only management
audit = AuditTask(
    device_ip="192.168.1.1",
    username="admin",
    password="secret",
    golden_rules=["ip ssh version 2", "no ip http server"]
)
print(audit.check("ip ssh version 2\nno ip http server\n"))
# []

```

4. Benefits of Network Automation

Benefit	Why it matters
Speed	Push changes to hundreds of devices in minutes instead of hours.

Benefit	Why it matters
Consistency	Same template applied everywhere; fewer human mistakes.
Auditability	Every command is logged in code and version control.
Scalability	Add 100 devices by updating a text file, not by hand.
Security	Reduce privileged manual access; enforce golden configs and rapid patching.
Recovery	Back up configs automatically and rollback quickly.

5. Network Automation Tools

5.1 Paramiko — Low-level SSH library

When to use: You need full control over the SSH session or want to build your own driver.

Workflow:

Create SSH client → Connect with credentials → Open shell/channel → Send commands → Read output

```
import paramiko
import time

class ParamikoDriver:
    def __init__(self, hostname, username, password):
        self.hostname = hostname
        self.username = username
        self.password = password
        self.client = paramiko.SSHClient()
        self.client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    def connect(self):
        self.client.connect(
            hostname=self.hostname,
            username=self.username,
            password=self.password,
            look_for_keys=False,
            allow_agent=False
        )

    def send_command(self, command):
        stdin, stdout, stderr = self.client.exec_command(command)
        return stdout.read().decode("utf-8")

    def close(self):
        self.client.close()
```

```
# Example: check uptime on a Linux jump host or open-gear console server
ssh = ParamikoDriver("192.168.1.10", "admin", "secret")
```

```
ssh.connect()
print(ssh.send_command("uptime"))
ssh.close()
```

Trade-off: Powerful but verbose; you handle prompts, paging, and vendor quirks yourself.

5.2 Netmiko — SSH for network devices

When to use: You want Paramiko-like SSH but with network-device awareness built in.

Workflow:

Pick device_type → Connect → send_command() / send_config_set() → Disconnect

```
from netmiko import ConnectHandler

class NetmikoDriver:
    def __init__(self, device_params):
        self.device_params = device_params
        self.connection = None

    def connect(self):
        self.connection = ConnectHandler(**self.device_params)

    def get_version(self):
        return self.connection.send_command("show version")

    def configure_acl(self, acl_lines):
        return self.connection.send_config_set(acl_lines)

    def save(self):
        return self.connection.save_config()

    def disconnect(self):
        if self.connection:
            self.connection.disconnect()

# Example: Cisco IOS device
ios_device = {
    "device_type": "cisco_ios",
    "host": "192.168.1.1",
    "username": "admin",
    "password": "secret",
    "secret": "enable_secret", # for enable mode
}

net = NetmikoDriver(ios_device)
net.connect()
print(net.get_version())
net.configure_acl([
    "ip access-list extended BLOCK_BAD",
    "deny ip 10.0.0.0 0.255.255.255 any",
    "permit ip any any"
])
net.save()
net.disconnect()
```

Trade-off: Easier than Paramiko for CLI-based devices; less flexible for non-network SSH targets.

5.3 Ansible — Agentless automation framework

When to use: You need declarative, idempotent playbooks and want to manage many devices from a control node.

Workflow:

Inventory → Playbook (YAML) → Modules (ios_command, nxos_config, ...) → Report

```
# inventory.ini
[cisco_routers]
192.168.1.1 ansible_network_os=ios ansible_user=admin ansible_password=secret

# backup.yml
- name: Backup router configs
  hosts: cisco_routers
  gather_facts: no
  tasks:
    - name: Run show running-config
      ios_command:
        commands: show running-config
      register: config_output

    - name: Save to file
      copy:
        content: "{{ config_output.stdout[0] }}"
        dest: "backups/{{ inventory_hostname }}.cfg"

# You can also drive Ansible from Python when needed
import ansible_runner

class AnsibleRunner:
    def __init__(self, playbook_path, inventory_path):
        self.playbook_path = playbook_path
        self.inventory_path = inventory_path

    def run(self):
        result = ansible_runner.run(
            private_data_dir=".",
            playbook=self.playbook_path,
            inventory=self.inventory_path
        )
        return result.status, result.rc

# Example
runner = AnsibleRunner("backup.yml", "inventory.ini")
print(runner.run())
```

Trade-off: Great for standardisation and teams; YAML can become complex for heavy logic.

5.4 NAPALM — Multivendor network abstraction library

When to use: You want the same Python API across Cisco, Juniper, Arista, and other vendors.

Workflow:

Get driver for vendor → Open connection → Load merge/replace config →

Commit / Discard

```
from napalm import get_network_driver

class NapalmDriver:
    def __init__(self, vendor, hostname, username, password):
        driver = get_network_driver(vendor)
        self.device = driver(hostname, username, password)

    def connect(self):
        self.device.open()

    def load_and_merge(self, config_file):
        self.device.load_merge_candidate(filename=config_file)
        return self.device.compare_config()

    def commit(self):
        self.device.commit_config()

    def rollback(self):
        self.device.discard_config()

    def disconnect(self):
        self.device.close()

# Example: safely merge a snippet on a Cisco IOS router
napalm = NapalmDriver("ios", "192.168.1.1", "admin", "secret")
napalm.connect()
diff = napalm.load_and_merge("snippets/ntp.cfg")
print(diff) # Review before commit
if "ntp server" in diff:
    napalm.commit()
else:
    napalm.rollback()
napalm.disconnect()
```

Trade-off: Cleaner than raw SSH for config management; not every feature is abstracted.

5.5 Telnet — Legacy plain-text access

When to use: Only when the target device does not support SSH and there is no other option.

Workflow:

Open TCP socket to port 23 → Send username/password → Send commands → Read output

```
import telnetlib
import getpass

class TelnetDriver:
    def __init__(self, hostname, username, password, timeout=5):
        self.hostname = hostname
        self.username = username
        self.password = password
        self.timeout = timeout
        self.tn = None

    def connect(self):
```

```

self.tn = telnetlib.Telnet(self.hostname, timeout=self.timeout)
self.tn.read_until(b"Username: ", self.timeout)
self.tn.write(self.username.encode() + b"\n")
self.tn.read_until(b"Password: ", self.timeout)
self.tn.write(self.password.encode() + b"\n")

def send_command(self, command):
    self.tn.write(command.encode() + b"\n")
    time.sleep(1)
    return self.tn.read_very_eager().decode("utf-8", errors="ignore")

def close(self):
    if self.tn:
        self.tn.close()

# Example: collect a show command from a legacy lab switch
import time
legacy = TelnetDriver("192.168.1.99", "admin", "secret")
legacy.connect()
print(legacy.send_command("show version"))
legacy.close()

```

⚠ **Security warning:** Telnet sends credentials and commands in plain text. Prefer SSH in production.

6. Paramiko vs Netmiko

Feature	Paramiko	Netmiko
Level	Low-level SSH library	High-level network SSH wrapper
Device awareness	None — generic SSH	Knows Cisco IOS, JunOS, NX-OS, Arista, etc.
Paging handling	You must send terminal length 0 yourself	Handles terminal length 0 automatically for many vendors
Config mode	Manual enable / configure terminal	Built-in enable() and send_config_set()
Error detection	Manual	Better parsing of command errors and prompts
Use case	Linux servers, custom protocols, tunnels	CLI-based routers, switches, firewalls

Quick decision tree

```

Target is a Linux/BSD server and you need raw SSH?
→ Paramiko
Target is a router/switch/firewall CLI?
→ Netmiko
Need declarative fleet-wide automation?
→ Ansible
Need multivendor config diffs and rollback?

```

→ NAPALM
Only legacy device supports Telnet?
→ Telnetlib (lab only, prefer SSH)

7. Best Practices

Practice	Why
Use SSH, not Telnet	Encrypts credentials and data in transit.
Store secrets in vaults	Keep passwords and keys out of source code (use environment variables, HashiCorp Vault, Ansible Vault).
Back up before change	Always capture <code>show running-config</code> before a merge or replace.
Test in a lab first	Validate playbooks and scripts on non-production gear.
Make changes idempotent	Running the script twice should not break anything.
Parse with templates	Use TextFSM, Jinja2, or NAPALM getters to turn CLI blobs into structured data.
Version-control everything	Track inventory, templates, and playbooks in Git.
Log and audit	Record who ran what, when, and on which devices.

Idempotent config example with Netmiko

```
class SafeConfigManager(NetmikoDriver):
    def apply_if_missing(self, required_line, all_lines):
        if required_line not in all_lines:
            return self.connection.send_config_set([required_line])
        return f"Already present: {required_line}"

# Applying the same NTP server twice is safe
safe = SafeConfigManager(ios_device)
safe.connect()
config = safe.connection.send_command("show running-config | include ntp")
print(safe.apply_if_missing("ntp server 10.0.0.1", config))
print(safe.apply_if_missing("ntp server 10.0.0.1", config)) # no duplicate
safe.disconnect()
```

8. Future of Network Automation

Trend	What it means
Intent-based networking (IBN)	State the goal (“allow web traffic”), system figures out the config.

Trend	What it means
API-first devices	RESTCONF/NETCONF replace screen-scraping CLI for safer, structured changes.
Event-driven automation	Tools like StackStorm or custom webhooks react to syslog/SNMP traps.
AI/ML-assisted ops	Anomaly detection, predictive failure analysis, and auto-remediation.
GitOps for networks	Device configs live in Git; CI/CD pipelines validate and deploy them.

9. Cheat Sheet

Job	Tool of choice
Raw SSH to Linux/servers	Paramiko
SSH to routers/switches	Netmiko
Declarative fleet playbooks	Ansible
Multivendor config merge/diff	NAPALM
Legacy plain-text only	Telnetlib (lab)
Structured data from CLI	TextFSM + Netmiko

10. Minimal Lab Checklist

- ☐ One control machine with Python 3, paramiko, netmiko, napalm, ansible installed.
- ☐ One or more network devices (physical, GNS3, EVE-NG, or Cisco CML).
- ☐ A devices.yml inventory file.
- ☐ A backup playbook before any change.
- ☐ A rollback plan for every pushed config.

```
# Tiny inventory loader for the examples above
import yaml
```

```
class Inventory:
    def __init__(self, path):
        with open(path) as f:
            self.devices = yaml.safe_load(f)

    def by_vendor(self, vendor):
        return [d for d in self.devices if d.get("device_type") == vendor]
```

```
# devices.yml example:
```

```
# - device_type: cisco_ios
#   host: 192.168.1.1
#   username: admin
#   password: secret
```

Remember: Network automation is not about replacing engineers — it is about removing repetitive, error-prone work so you can focus on architecture, security, and reliability.